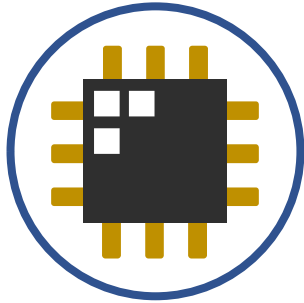




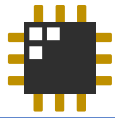
EEE 113:

Introduction to Electrical and Electronics Engineering Systems

Week 4 – Assembly programming, Hardware/Software Interface

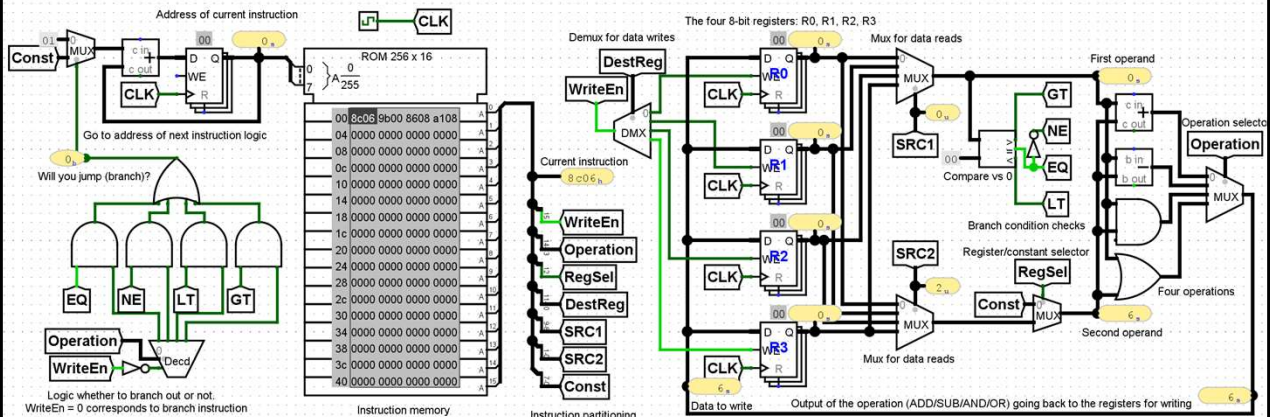
1

Welcome to Week 4. For this week, we will be focusing on assembly programming. The previous lecture covered how we would architect our processor given some instructions. Now, we will focus on implementing more complex programs that can run on the processor. These assembly programming exercises will serve as a motivation for the need for high-level programming languages that would make code writing more efficient and easier to read/follow. We will also introduce I/O interfacing which would complete a computer system that can interact or send some feedback to the user.



Recap: Your first processor ☺

- 16-bit instructions, 8-bit data, 4 registers, 4 operations, 4 branch conditions

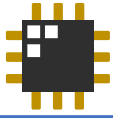


This is a *custom processor* made for EEE 113. Simple enough to be understood. Complex enough to implement simple programs. Previous EEE 113 offerings had 'Machine Architecture that is Really Intuitive and Easy' (MARIE) Now, for this offering, we have 'Architecture Design that's Easy to Learn' (ADEL) ☺

This is the processor that we have created in the last lecture. This has 16-bit instructions where each bit plays a role in how the processor runs. There are 4 registers in this processor and each register contains 8-bits each. It can support 8-bit constants as the second operand. It has four arithmetic-based operations: ADD/SUB/AND/OR. For branches, it supports up to 4 different conditions: EQ/NE/LT/GT.

The 16-bit instructions are acquired from the instruction memory and then partitioned into the corresponding fields that would be used by the processor to know what to do. The register operands are then read from the register file, a collection of registers (4 registers in this case) that has a multiplexer to select which of the registers are relevant in the operation. The second operand has an additional multiplexer to select whether the operation needs a register value or a constant. Once the operands are prepared, the actual operation is then performed as the two operands go through the different operations (4 in this case: ADD/SUB/AND/OR) which will be multiplexed again selecting only the relevant operation needed by the instruction. As the result of the operation is calculated, it is then written back to the register file depending on the destination register using a demultiplexer. That completes the arithmetic and logic instruction implementation.

Branch instructions, which are instructions that allow for a change in program flow, are executed a little differently. The 2-bit operation code is used to identify the branch condition that needs to be satisfied. This is done through a decoder (left side of the drawing) that is enabled only if the instruction will not write to the register file. The first source operand is checked by a comparator to identify whether it is equal, less than, or greater than 0. This is then used to identify whether the branch condition is indeed satisfied or not. The branch decoding logic would act as a selector for a multiplexer that selects whether the next instruction is address + 1 or address + constant. That completes the branch instruction implementation.



Recap: Talking to your processor

• The instruction encoding that the processor supports

W	OPC	RS	DEST	SRC1	CONST/SRC2
	If W = 0:		If W = 1:	00 – R0	If RS = 0: The second operand is 8-bit CONST This is written in two's complement
	00 – EQ		00 – R0	01 – R1	
	01 – NE		01 – R1	10 – R2	
	10 – LT		10 – R2	11 – R3	
	11 – GT		11 – R3		
	If W = 1:		If W = 0:		If RS = 1: Second operand is the last 2-bits SRC2
	00 – ADD		00 – branches do not write		0000 0000 – R0
	01 – SUB				0000 0001 – R1
	10 – AND	If W = 0:			0000 0010 – R2
	11 – OR	0 – branches have no second operand			0000 0011 – R3
		If W = 1:			
		0 – Constant second operand (ADDI/SUBI/ANDI/ORI)			
		1 – Register second operand (ADD/SUB/AND/OR)			

Sample inst

ADD R0,

SUBI R3,

BNE R2,

Sample instructions:

```
ADD R0, R2, R3
SUBI R3, R1, -31
BNE R2, -2
```

0: BEQ/BNE/BLT/BGT (Instructions that do not write to the register file)

1: ADD/SUB/AND/OR and ADDI/SUBI/ANDI/ORI (Instructions that write to the register file)

Here's the instruction encoding that includes all the instructions supported by the processor.

W determines whether we write to the register file (1) or not (0). Only branch instructions don't write to the register file, W = 0. Arithmetic and logic instructions: ADD/SUB/AND/OR and their immediate/constant second operand counterparts ADDI/SUBI/ANDI/ORI will write to the register file so W = 1.

OPC changes depending if the instruction is a branch (mapped to the branch conditions: 00 for EQ, 01 for NE, 10 for LT, 11 for GT) or if the instruction is a typical arithmetic instruction (mapped to the operation codes: 00 for ADD, 01 for SUB, 10 for AND, 11 for OR).

RS determines if the second operand is a constant (0) or a register (1). It is not relevant for branches (0).

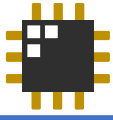
DEST determines the register mapping for the destination, the register that will be updated (00 for R0, 01 for R1, 10 for R2, 11 for R3). It is not relevant for branches (00).

SRC1 determines the register mapping of the first operand. It follows the same binary mapping for the register names.

CONST/SRC2 is treated as the 8-bit two's complement constant if RS = 0 or for branches. If RS = 1, then it maps the second register operand of the instruction

(last 2 bits), padded with six 0s before it.

Some sample instructions are also written as a guide, with different fields colored accordingly with the instruction encoding.



Assembly programming

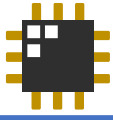
- The processor is complex enough to implement *cool* programs
 - Break down algorithms into instructions that the processor *understands*
 - Step-by-step instructions!

Let's have some assembly programming exercises!

4

Now that we have built the processor, let's try to run some interesting programs in it. As long as the algorithms that we will implement can fit using only 4 registers (since the processor only supports 4 registers) and using only the supported operations ADD/SUB/AND/OR, then we should be able to run it on the processor.

The key to assembly programming is to break down the algorithms into a series of instructions. Since instructions can only do one operation at a time, we have to write our algorithms in a step-by-step manner.



Assembly programming example

- Multiplication through repeated addition: **R2 = R0 x R1**

ADDI R0, R0, 15

ADDI R1, R1, 7

BEQ R1, 4

loop:

ADD R2, R2, R0

SUBI R1, R1, 1

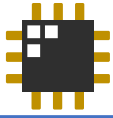
BNE R1, -2

end:

BEQ R1, 0

What does each line do?

Here's a sample assembly code that implements the multiplication algorithm using repeated addition. R0 and R1 contain the operands for the multiplication and the result is stored in R2. Let's try to identify what each line of the code does and how they play a role in implementing the desired algorithm.



Assembly programming example

- Multiplication through repeated addition: **R2 = R0 x R1**

ADDI R0, R0, 15 *# first number to multiply*

ADDI R1, R1, 7 *# second number to multiply*

BEQ R1, 4 *# end, you're multiplying with 0*

loop:

ADD R2, R2, R0 *# repeated addition, R2 gets the product*

SUBI R1, R1, 1 *# reduce this until it reaches 0*

BNE R1, -2 *# back to loop*

end:

BEQ R1, 0 *# infinite loop to halt program*

6

We first assume that all registers start with a value of 0.

The first line sets R0 to be equal to 15 ($R0 = R0 + 15$), which will be the first number that we will multiply.

The second line sets R1 to be equal to 7 ($R1 = R1 + 15$), which will be the second number that we will multiply.

The third line checks if R1 is already equal to 0. If it is indeed 0, then we just bypass the multiplication process since we are multiplying with 0. We will jump all the way to the 'end' label of the code which is 4 instructions away from the current instruction (that's why the constant is 4). R2, which should contain the product, has an initial value of 0, so the product is already correct.

If the previous condition is not true, then we proceed to the main multiplication loop. The fourth instruction essentially accumulates the product on R2 by adding the value of R0 for every loop iteration.

The fifth instruction decrements the value of R1 by 1 for every loop iteration.

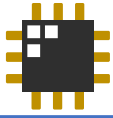
This serves as the counter on how many times are we going to add R0 into R2.

The sixth instruction checks if R1 is not equal to 0 yet. If it is not yet 0, then we continue the loop for another iteration. We go back to 2 instructions from the current instruction (that's why the constant is -2).

Once R1 becomes 0, then we will proceed to the last instruction which is

essentially an infinite loop to itself. R1 is already 0 when we reach this point. Branching to 0 instructions from the current instruction basically points to itself, thus creating the infinite loop. This can serve as our program halt for this example.

For this algorithm, we start accumulating the product on R2 while continuously decrementing R1 until it reaches 0. This is how we implement multiplication using repeated addition.



Assembly programming example

- Multiplication through repeated addition: **R2 = R0 x R1**

ADDI R0, R0, 15

0x800F

ADDI R1, R1, 7

0x8507

BEQ R1, 4

0x0104

loop:

ADD R2, R2, R0

0x9A00

SUBI R1, R1, 1

0xA501

BNE R1, -2

0x21FE

end:

BEQ R1, 0

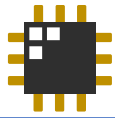
0x0100

The processor only understands 0s and 1s!

You can use the Python assembler to convert instructions into machine code!

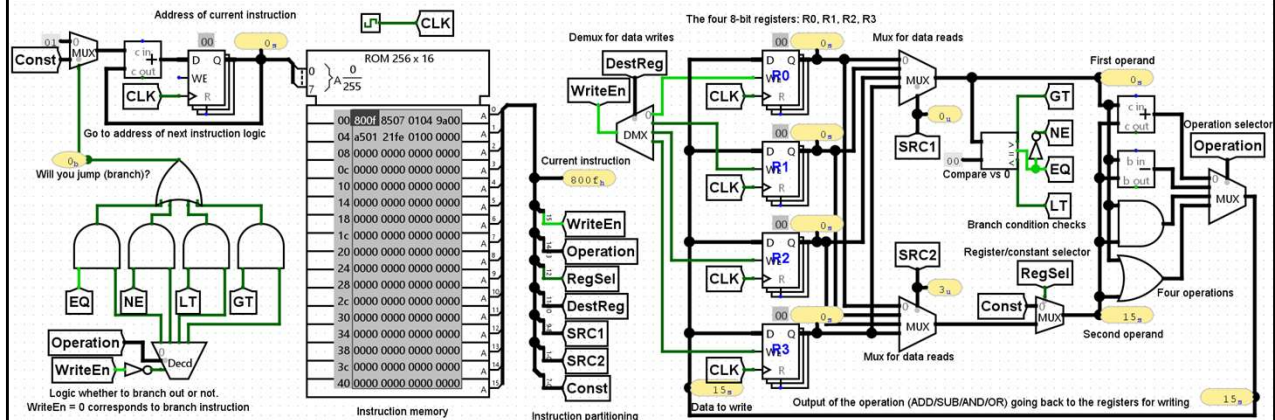
7

For the processor to run these instructions, they have to be converted into machine code first. That is, the instructions must be converted into strings of 0s and 1s just like how the processor functionality was demonstrated in the previous lecture. You can use the provided assembler, written in Python, to automatically convert these instructions into their corresponding 16-bit binary strings (which will be written in hex). You can also manually verify the instruction to machine code conversion by following the instruction encoding table that was shown in the first few slides of this lecture. The hex values shown here should be equivalent to what you will end up deriving.



Assembly programming example

- Multiplication through repeated addition: **$R2 = R0 \times R1$**



8

"ADDI R0, R0, 15", # Load initial value of R0 = 15, $R0 = R0 + 15$

"ADDI R1, R1, 7", # Load initial value of R1 = 7, $R1 = R1 + 7$

"BEQ R1, 4", # Check if R1 = 0, if it is, exit (go to end)

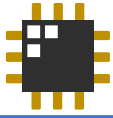
"ADD R2, R2, R0", # $R2 = R2 + R0$ (R2 is the accumulator)

"SUBI R1, R1, 1", # $R1 = R1 - 1$ (reduce this until it reaches 0)

"BNE R1, -2", # Loop back 3 steps backward

"BEQ R1, 0" # Infinite loop to halt operation

Here's a demonstration of the multiplication program running on the processor. We can see how R0 and R1 were initialized to 15 and 7 respectively. Then, we can see the several loop iterations until it reaches the end of the program. During the loop iterations, we can see the value of R2 being incremented by R0 and the value of R1 decrementing by 1 every time. After all the iterations, we can see that R2 will contain the product, which is 105 (the result of 15×7) and R1 would end up being equal to 0. This demonstrates the correct functionality of the multiplication algorithm using repeated addition as well as the capability of the processor to implement such an algorithm using the resources it has.



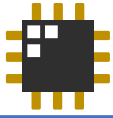
Assembly programming exercise 1

- Write an assembly program to perform **$R0 / R1 = R2$ remainder $R0$**
 - Assume **$R1$** will never be 0

9

Now that we've seen how much complexity the processor can support, let's try to write our own assembly code using a different algorithm.

For this exercise, we will implement a division algorithm using repeated subtraction. $R0$ is the numerator and $R1$ is the denominator. The quotient should end up placed in $R2$, while the remainder will be placed in $R0$.



Assembly programming exercise 1

- Write an assembly program to perform **$R0 / R1 = R2$ remainder $R0$**
 - Assume **$R1$** will never be 0

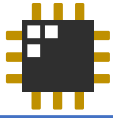
Try writing a pseudocode on what to do step-by-step:

- 1) Put operands on $R0$ and $R1$
- 2) Create a loop, check if $R0 < R1$ to exit
- 3) Inside the loop: increment $R2$, $R0 = R0 - R1$
- 4) *Create an infinite loop at the 'exit' to halt the program*

(not really required, but it's a nice practice to 'stop' the processor for now)

10

Every time you write a code, it is beneficial to start writing some pseudocode first to have an idea of what needs to be done. To implement this algorithm, we first set $R0$ and $R1$ to their initial values, the numerator and denominator respectively. Then we will have to create a loop that performs the repeated subtraction process. We will continue on looping until $R0 < R1$. Within the loop, we will continuously perform the repeated subtraction process $R0 = R0 - R1$ ($R0$ will inevitably contain the remainder), while incrementing $R2$ (which will be the quotient) by 1 every iteration. We will stay in this loop until $R0 < R1$, after which we exit the loop and enter an infinite loop to halt the program.



Assembly programming exercise 1

- Write an assembly program to perform **$R0 / R1 = R2$ remainder $R0$**

$R0$ = numerator

$R1$ = denominator

loop:

if ($R0 < R1$), go to *exit*

else, $R2 = R2 + 1$

$R0 = R0 - R1$

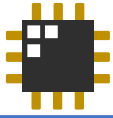
Go back to *loop*

***exit:* infinite loop here**

Convert to assembly!

11

And so, writing the pseudocode into a form that somewhat resembles an assembly program, we have these instructions. The next step is to actually write the corresponding assembly code to perform these operations.



Assembly programming exercise 1

- Write an assembly program to perform **R0 / R1 = R2 remainder R0**

```
ADDI R0, R0, 17      # R0 = numerator
ADDI R1, R1, 5       # R1 = denominator
loop:
    SUB R3, R0, R1    # Check if R0 < R1
    BLT R3, exit      # Go to exit if indeed R0 < R1
    ADDI R2, R2, 1    # Increment R2
    SUB R0, R0, R1    # R0 = R0 – R1
    BGT R1, loop      # Go back to loop
exit: BLT R3, exit    # infinite loop to 'stop' program
```

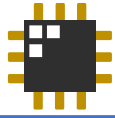
12

Here's the assembly equivalent of the pseudocode that we had earlier. Almost everything is consistent with the pseudocode except for the check of $R0 < R1$. Note that the processor can only perform comparisons against 0 (recall the different branch conditions: EQ = equal to 0; NE = not equal to 0; LT = less than 0; GT = greater than 0). We cannot perform a condition check for $R0 < R1$. Instead, we can perform a subtraction first, $R0 - R1$, and store the result on another register, in this case, R3. Then, we can check the value of this register. If $R0 < R1$, then $R0 - R1$ will end up being a negative number. Thus, we can check the value of R3 (the container for $R0 - R1$) and check if it is a negative number. If it is negative already, then $R0 < R1$, we exit the loop as written in the pseudocode that we had earlier.

We also had to write a branch instruction that allows us to go back to the loop label. The pseudocode indicates that the jump to the loop label does not need to be checked for a condition. However, our processor needs some conditions to check every time it runs a branch instruction. We can simply think of any condition that will always be true for our purposes. The second to the last instruction (BGT R1, loop) would always be true as long as the value R1 is greater than 0. Assuming we are performing the division process correctly, R1 should always be greater than 0. Thus, BGT R1, loop will always end up

jumping to the loop label.

Finally, we implement the infinite loop by simply checking if R3 is negative. This condition will always be true since we will only end up at this instruction when we exit the repeated subtraction loop. Thus, R3 will always be negative once we exit that loop. Consequently, the last instruction (BLT R3, exit) will always be true and will cause the infinite loop that we desire.



Assembly programming exercise 1

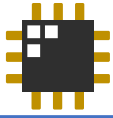
- Write an assembly program to perform **R0 / R1 = R2 remainder R0**

```
ADDI R0, R0, 17      # R0 = numerator
ADDI R1, R1, 5       # R1 = denominator
loop:
  SUB R3, R0, R1      # Check if R0 < R1
  BLT R3, 4          # Go to exit if indeed R0 < R1
  ADDI R2, R2, 1      # Increment R2
  SUB R0, R0, R1      # R0 = R0 - R1
  BGT R1, -4         # Go back to loop
exit: BLT R3, 0       # infinite loop to 'stop' program
```

13

We should remember, however, that branch instructions require actual constants which indicate where the target instruction would be if we are jumping.

Remember that branch constants are relative from where you are to where you want to go. Thus, for the fourth instruction (**BLT R3, exit**), the exit label is 4 instructions away from the current instruction. Therefore, we write 4 as the constant for that. The seventh instruction (**BGT R1, loop**) points to the loop label which is 4 instructions going back. Therefore, we write -4 as the constant for that. Finally, the last instruction, which is an infinite loop, would point to the same instruction (itself), thus the constant will be 0. This slide shows the conversion of the labels into their corresponding constants by following the relative offsets needed to jump to the intended label.



Assembly programming exercise 1

- Write an assembly program to perform **R0 / R1 = R2 remainder R0**

ADDI R0, R0, 17 *0x8011*

ADDI R1, R1, 5 *0x8505*

loop:

SUB R3, R0, R1 *0xBC01*

BLT R3, 4 *0x4304*

ADDI R2, R2, 1 *0x8A01*

SUB R0, R0, R1 *0xB001*

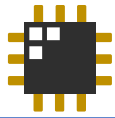
BGT R1, -4 *0x61FC*

exit: **BLT R3, 0** *0x4300*

Let's run this now!

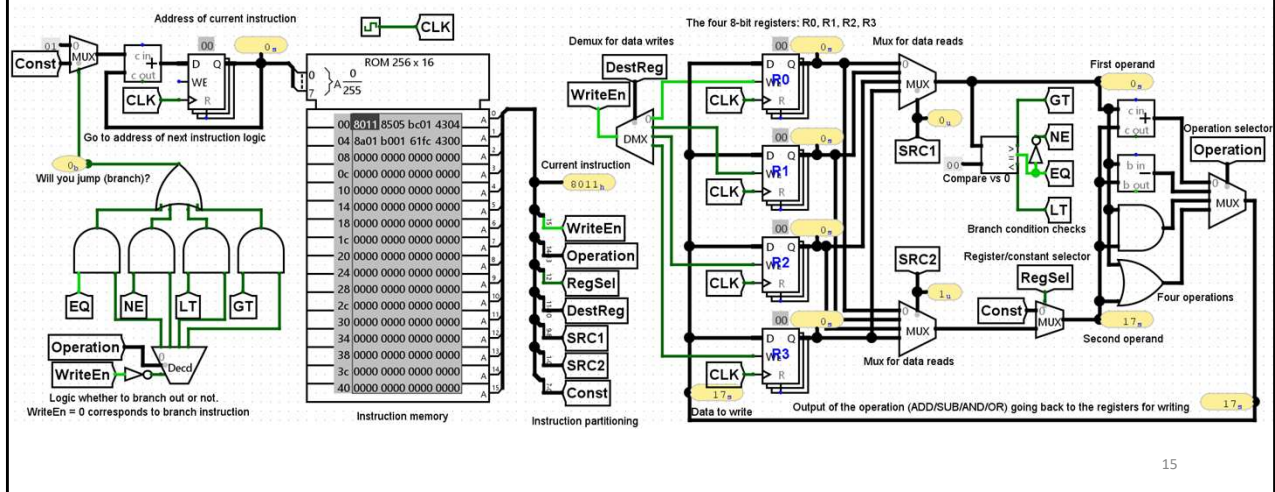
14

Once the instructions are ready, we can proceed to convert the instructions into their machine code equivalent. Once converted, they can now be loaded to the processor for simulation.



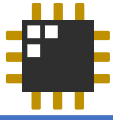
Assembly programming exercise 1

- Write an assembly program to perform $R0 / R1 = R2$ remainder $R0$



```
"ADDI R0, R0, 17", # Load initial value of R0 = 17
"ADDI R1, R1, 5", # Load initial value of R1 = 5
"SUB R3, R0, R1", # Subtract R0 and R1 to check if R0 < R1
"BLT R3, 4", # Check if R0 < R1, if it is, exit (go to end)
"ADDI R2, R2, 1", # Update the quotient, R2 = R2 + 1
"SUB R0, R0, R1", # Update the remainder, R0 = R0 - R1
"BGT R1, -4", # Loop back 3 steps backward
"BLT R3, 0" # Infinite loop to halt operation
```

Loading the machine code from the previous slide into the processor, we have this demonstration. We can see how the values of R0 and R1 are initialized to the numerator and denominator, respectively. Then we will proceed to iterate several times inside the loop to continuously perform $R0 = R0 - R1$ for the repeated subtraction process, while incrementing R2 by 1 every iteration. By the end of all the iterations, R2 would contain the quotient of the division algorithm, while R0 would end up containing the remainder. This completes the division using repeated subtraction implementation demonstrated on the processor.

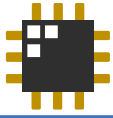


Assembly programming exercise 2

- Write an assembly program to increment R3 from 0 to 9, then back to 0
 - This is an infinite loop. R3 just counts up then loops back to 0 (0000) after reaching 9 (1001)

16

Let's have another exercise. For this one, we will write a program that would continuously increment the value of R3 from 0 to 9 and then back to 0 again. This would be implemented as an infinite loop.



Assembly programming exercise 2

- Write an assembly program to increment R3 from 0 to 9, then back to 0
 - This is an infinite loop. R3 just counts up then loops back to 0 (0000) after reaching 9 (1001)

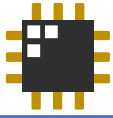
Try writing a pseudocode on what to do step-by-step:

- 1) Increment R3: $R3 = R3 + 1$
- 2) If $R3 \neq 9$, go back to step 1. Otherwise, proceed.
- 3) Set $R3 = 0$, then go back to 1.

17

As always, we try to write the pseudo code first to get an idea of what needs to be done.

For this program, the idea is simple: we create a loop incrementing R3 by 1 every iteration, then check if R3 equals 9 or not. If it is not equal to 9, then just loop again to increment R3. However, if R3 is indeed equal to 9, then R3 is set to 0 first before starting the incrementing process again.



Assembly programming exercise 2

- Write an assembly program to increment R3 from 0 to 9, then back to 0
 - This is an infinite loop. R3 just counts up then loops back to 0 (0000) after reaching 9 (1001)

loop:

R3 = R3 + 1

if (R3 != 9), go to *loop*

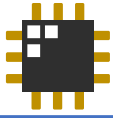
R3 = 0

go to *loop*

Convert to assembly!

18

Writing the pseudo-code into a form that resembles an assembly code, we end up with the following. Now, the next task is to write the actual instructions needed to implement this.



Assembly programming exercise 2

- Write an assembly program to increment R3 from 0 to 9, then back to 0
 - This is an infinite loop. R3 just counts up then loops back to 0 (0000) after reaching 9 (1001)

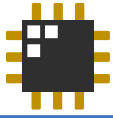
loop:

ADDI R3, R3, 1	# Increment R3
SUBI R2, R3, 9	# Check if R3 != 9
BNE R2, loop	# If it is, go back to loop
SUB R3, R3, R3	# Else, set R3 = 0
BEQ R3, loop	# Go back to loop

19

Most of the instructions are straightforward. We again note that checking for the condition of $R3 \neq 9$ cannot be done in a single instruction because the branch instructions supported by the processor only allow for comparisons against 0. Thus, employing the same technique we did for the division algorithm, we first use another register, R2 in this case, to contain the value of $R3 - 9$. Then, if R3 is not equal to 9, then $R2 = R3 - 9$ will not be equal to 0. We can then use this logic to write the branch instruction: **BNE R2, loop**.

The final loop in our code is unconditional, meaning we don't really have to check for a condition, we just jump right away. However, branch instructions supported by the processor always need to check for conditions. Thus, we can just leverage the fact that R3 would always be equal to 0 to reach this point. Therefore, we can write **BEQ R3, loop** to create a loop that is always true given the context of this program code.



Assembly programming exercise 2

- Write an assembly program to increment R3 from 0 to 9, then back to 0
 - This is an infinite loop. R3 just counts up then loops back to 0 (0000) after reaching 9 (1001)

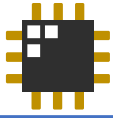
loop:

ADDI R3, R3, 1
SUBI R3, R3, 9
BNE R2, [-2]
SUB R3, R3, R3
BEQ R3, [-4]

Increment R3
Check if R3 != 9
If it is, go back to loop
Else, set R3 = 0
Go back to loop

20

Of course, we have to write the corresponding constants for the branch instructions. The third instruction BNE R2, loop is pointing to the loop label (which is the very first instruction of the code) which is 2 instructions going back from the current instruction. Thus, we put -2 as the constant. The fifth instruction BEQ R2, loop also points to the loop label which is now farther away: 4 instructions going back. Thus, we put -4 as the constant for this last instruction. After this point, we have completely written the true instruction format ready to be run on the processor.



Assembly programming exercise 2

- Write an assembly program to increment R3 from 0 to 9, then back to 0
 - This is an infinite loop. R3 just counts up then loops back to 0 (0000) after reaching 9 (1001)

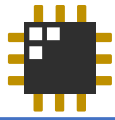
loop:

ADDI R3, R3, 1	0x8F01
SUBI R2, R3, 9	0xAB09
BNE R2, -2	0x22FE
SUB R3, R3, R3	0xBF03
BEQ R3, -4	0x03FC

Let's run this now!

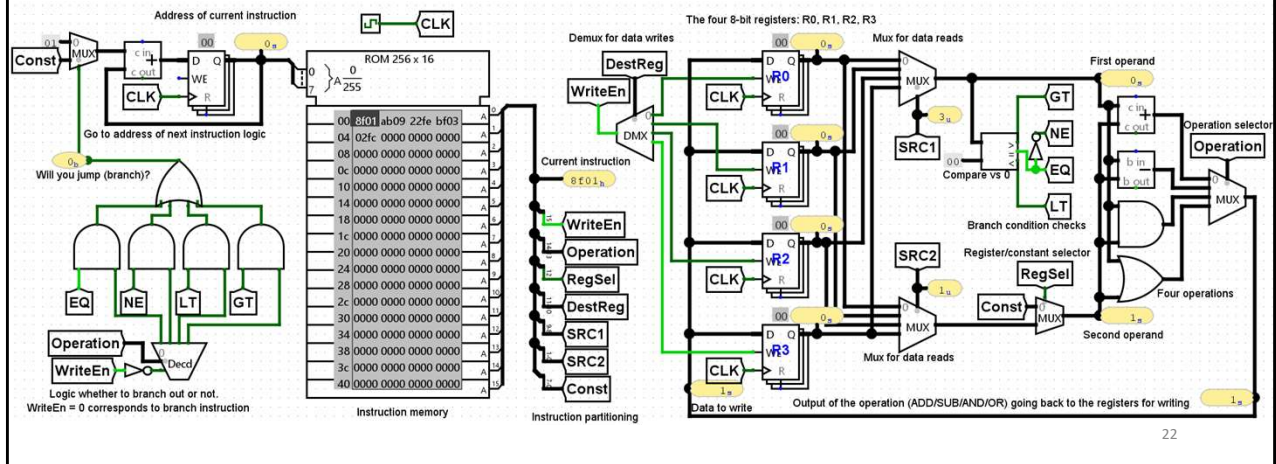
21

Before loading the instructions into the processor, we have to convert the instructions into machine code first. Again, you can manually verify these by checking the instruction encoding that defines how each instruction should be written in its binary string counterpart.



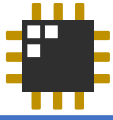
Assembly programming exercise 2

- Write an assembly program to increment R3 from 0 to 9, then back to 0
 - This is an infinite loop. R3 just counts up then loops back to 0 (0000) after reaching 9 (1001)



```
"ADDI R3, R3, 1", # Increment R3 (R3 = R3 + 1)
"SUBI R2, R3, 9", # Check if R3 equals 9 (we're at the limit), store it at R2
"BNE R2, -2",    # If R3 != 9 (R2 = R3 - 9, R2 != 0), we're good, just loop back
"SUB R3, R3, R3", # If R3 == 9 (R2 = R3 - 9, R2 == 0), reset R3 back to 0 (R3 = R3-R3)
"BEQ R3, -4"     # This creates an infinite loop back to the starting point (R3 = 0 at this point)
```

Here's the corresponding demonstration of the code on the processor. We can see how R3 is being incremented on an iteration of the loop. When R3 finally reaches 9, the first branch condition fails and R3 is reset back to 0 by subtracting it to itself. Afterward, we go back to the loop again to increment R3. This loop continues indefinitely.



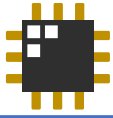
Assembly programming exercise 3

- Write the assembly code that gives out the nth Fibonacci number (stored in R0) and stores the result in R3.

23

Let's have one more exercise. This time a Fibonacci sequence generator. For this exercise, we will write a code that will generate the nth Fibonacci number (R0 has the value n) and store the result in R3.

Fibonacci sequence: 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, and so on. The next number in the series is the sum of the two numbers preceding it.



Assembly programming exercise 3

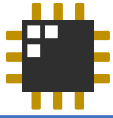
- Write the assembly code that gives out the nth Fibonacci number (stored in R0) and stores the result in R3.

Try writing a pseudocode on what to do step-by-step:

- 1) Set $R0 = n$, $R1 = 0$, $R2 = 1$, $R3 = R2$ (1st Fibonacci number)
- 2) Decrement R0. Then, check if $R0 == 0$, if true, exit prog
- 3) $R3 = R1 + R2$
- 4) $R1 = R2$
- 5) $R2 = R3$
- 6) Loop back to step 2

24

Writing the pseudo-code for this exercise, we have this. We first initialize the register values. R0 gets the value of n , while R1 and R2 will contain the two previous values of the Fibonacci series, which allow for the calculation of the next number in the series to be stored in R3. We can also initially set R3 to have the value of $R2 = 1$. Then, since R0 contains the target index of the Fibonacci sequence (we assume that R0 can only be greater than 0), we decrement R0 by 1 and then check if R0 already contains 0 to indicate that we have to exit the Fibonacci generator loop. Otherwise, we proceed to calculate the next number in the series by performing $R3 = R1 + R2$ (the next number is the sum of the last two), then update R1 and R2 to contain the last two values in preparation for the next sum generation: $R1 = R2$ and $R2 = R3$. Afterward, we loop back to the decrement R0 step until R0 becomes 0, completing the loop.



Assembly programming exercise 3

- Write the assembly code that gives out the nth Fibonacci number (stored in R0) and stores the result in R3.

R0 = R0 + n (say 8)

R2 = R2 + 1

R3 = R2

loop:

R0 = R0 - 1

if (R0 == 0), go to exit

else, R3 = R1 + R2

R1 = R2

R2 = R3

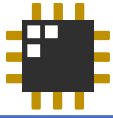
go to loop

exit: end of program (inf loop)

Convert to assembly!

25

Writing the pseudo-code into a form that somewhat resembles an assembly program, we have these. The first three steps are just initialization phases: setting R0 to n (in this case, 8 as an example), setting R2 = 1, and R3 = R2 = 1. Note that R0 is already initialized to R0. After the initialization phase, we proceed to the loop, where we decrement R0 by 1 and then check if R0 is already 0. If it is, then we stop the process and the loop ends. Otherwise, we proceed to calculating the next Fibonacci number by summing the last two numbers in the series $R3 = R1 + R2$ and then updating R1 and R2 in preparation for the next calculation. This completes the Fibonacci generator loop.

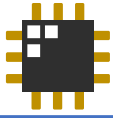


Assembly programming exercise 3

```
ADDI R0, R0, 8      # Initialize R0, n = 8
ADDI R2, R2, 1      # R2 = 1
ADDI R3, R2, 0      # R3 = R2
loop:
  SUBI R0, R0, 1     # Decrement R0
  BEQ R0, exit       # Check if R0 == 0; if it is, exit
  ADD R3, R1, R2     # R3 = R1 + R2 (Fibonacci)
  ADDI R1, R2, 0     # R1 = R2 (adjust 1st term)
  ADDI R2, R3, 0     # R2 = R3 (adjust 2nd term)
  BNE R0, loop       # loop back to loop label
exit: BEQ R0, exit    # infinite loop to 'halt' program
```

26

Writing the previous code in actual assembly, we have the following. Most of the instructions are straightforward. The second to the last instruction required an unconditional branch and, in this case, BNE R0, loop was used. Since the loop iteration will only be performed if R0 is not equal to 0 (due to the earlier check in BEQ R0, exit), then we are guaranteed that R0 is indeed not equal to 0 when we execute BNE R0, loop. At the exit marker, we also created an infinite loop to halt the program. This is simply implemented through BEQ R0, exit, since we are guaranteed that we will only reach this point if R0 is already equal to 0 (due to the earlier check in BEQ R0, exit).

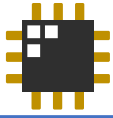


Assembly programming exercise 3

```
ADDI R0, R0, 8      # Initialize R0, n = 8
ADDI R2, R2, 1      # R2 = 1
ADDI R3, R2, 0      # R3 = R2
loop: SUBI R0, R0, 1  # Decrement R0
      BEQ R0, 5      # Check if R0 == 0; if it is, exit
      ADD R3, R1, R2  # R3 = R1 + R2 (Fibonacci)
      ADDI R1, R2, 0  # R1 = R2 (adjust 1st term)
      ADDI R2, R3, 0  # R2 = R3 (adjust 2nd term)
      BNE R0, -5      # loop back to loop label
exit: BEQ R0, 0       # infinite loop to 'halt' program
```

27

Of course, we have to convert the label references into actual constants that the processor will understand. Recall that the constants for the branch instructions should be the number of instructions you need to jump to relative to where the branch instruction currently is. Therefore, for the BEQ R0, exit instruction, the exit marker is 5 instructions forward from the current one, so we put the constant 5. Then, for the BNE R0, loop instruction, the loop marker is 5 instructions backward from where the current instruction is, so we put -5. Finally, for the BEQ R0, exit instruction, the exit marker points to the same instruction itself, therefore the constant should be 0.



Assembly programming exercise 3

ADDI R0, R0, 8 *0x8008*

ADDI R2, R2, 1 *0x8A01*

ADDI R3, R2, 0 *0x8E00*

loop:

SUBI R0, R0, 1 *0xA001*

BEQ R0, 5 *0x0005*

ADD R3, R1, R2 *0x9D02*

ADDI R1, R2, 0 *0x8600*

ADDI R2, R3, 0 *0x8B00*

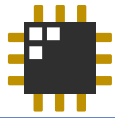
BNE R0, -5 *0x20FB*

exit: **BEQ R0, 0** *0x0000*

Let's run this now!

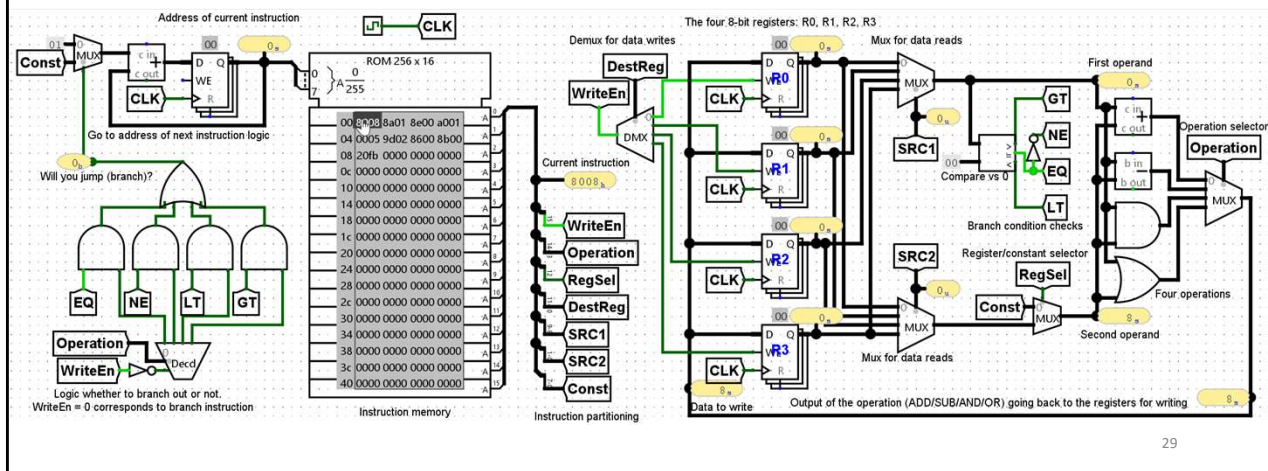
28

At this point, the assembly code is ready to be converted into machine code (binary string representation). We can run the provided Python assembler for this processor to do this. You can also manually verify these conversions, as a practice. You can do so since we know what the instruction encoding is, as presented in the first few slides of this lecture.



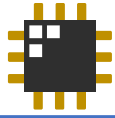
Assembly programming exercise 3

- Write the assembly code that gives out the nth Fibonacci number (stored in R0) and stores the result in R3.



```
"ADDI R0, R0, 8", # Load initial value of R0 = 8
"ADDI R1, R1, 0", # Load initial value of R1 = 0
"ADDI R2, R2, 1", # Load initial value of R2 = 1
"ADDI R3, R2, 0", # Set initial value of R3, the first Fibonacci number
"SUBI R0, R0, 1", # Decrement the Fibonacci number counter
"BEQ R0, 5",      # Check if R0 = 0, if it is, exit (go to end)
"ADD R3, R1, R2", # Fibonacci sequence is the sum of two previous values
"ADDI R1, R2, 0", # R1 gets R2 to move the two previous values
"ADDI R2, R3, 0", # R2 gets R3 to move the two previous values
"BNE R0, -5",     # Loop back 5 steps backward
"BEQ R0, 0"       # Infinite loop to halt operation
```

Running the instructions on the processor we have this demonstration. You can see how the values for R3, R2, and R1 change on every iteration of the loop. For this example, since R0 was initialized to 8 (thus, we are calculating for the 8th Fibonacci number), then the result should be 1, 1, 2, 3, 5, 8, 13, **21**. You can see the change in R3 going through all of these numbers and then stopping at 21, which is the point when R0 is already 0. This proves the correctness of both our algorithm and the processor implementation.



The hardware/software interface

- In the past, machine language was used
 - Imagine just writing a program using only binary/hex!

This can be too tedious!

ROM 256 x 16	
00	bf03 9000 9000 9000
04	9000 9000 8f01 c30f
08	a809 22fb a399 00f5
0c	8f07 23f5 0000 0000

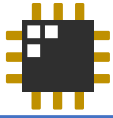
Imagine just writing this...



Computer scientist Margaret Hamilton poses with the Apollo guidance software that she and her team developed at MIT. Photos: MIT Museum

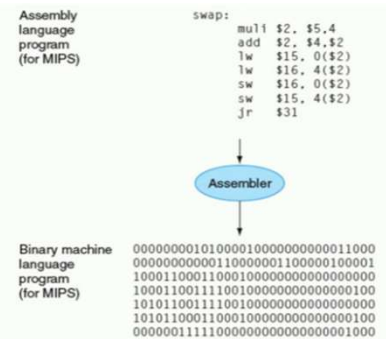
In the early days of computing, programmers used machine language, a low-level programming language that consists of binary or hexadecimal instructions that a computer can execute directly. Each type of computer has its own machine language. In our case, we had our own instruction encoding which corresponds to our specific processor design. Writing programs in machine language is a tedious task, as it requires programmers to know details about the hardware of the computer.

One of the earliest forms of machine language programming was done using punch cards, which were used to control looms in the 1800s. In the context of computing, punch cards were used to input data into computers in the 1960s and 1970s. Each card contained several punch holes representing data. These cards were fed into a card reader connected to a computer, which read the cards and stored the data in memory.

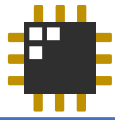


The hardware/software interface

- Eventually, symbolic notation was developed
 - An assembler translates assembly language to machine language
 - The assembler also computes for the relative offsets (branches for example!)
- However, assembly programming **was still tedious** since programmers have to think at the *machine level*
 - Step-by-step instructions
 - Code tends to be very long!
 - Syntax also makes the code hard to read the longer it becomes...

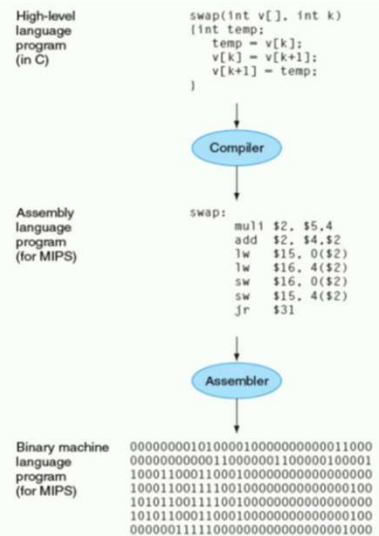
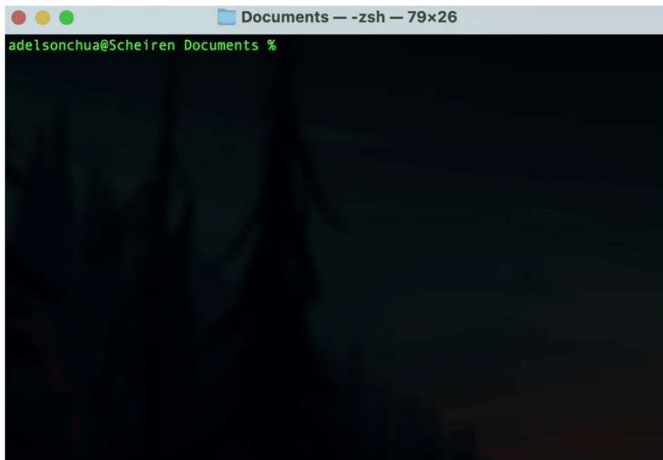


The transition from machine language to assembly language was a significant advancement in the field of computer science. Assembly language, while still low-level, is easier to read and write than machine language because it uses symbolic representations of machine instructions. Instructions are translated to their corresponding machine code through an assembler, which is essentially software mapping the instruction to the binary fields specific to the processor. Like machine language, assembly language is also hardware-specific. We need to know what instructions the processor supports. Moreover, writing in assembly can still be tedious since we have to write algorithms in a step-by-step manner. Consequently, the code tends to be very long for complex algorithms. Also, the way instructions are written also makes the code hard to read the longer the code is.



The hardware/software interface

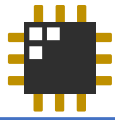
- Compilers were later developed allowing the use of high-level language to write software



The development of higher-level languages like C, Java, Python, etc., has made programming more accessible and less error-prone, as these languages are easier to read and write and are not tied to specific hardware. Compilers are used to convert these high-level programs into the corresponding assembly program. Compilers are also software that analyzes patterns in the high-level code and translates those into their corresponding assembly program. High-level language makes programming significantly easier due to the syntax and introduction of high-level constructs, making the code shorter and easier to understand. Moreover, it makes the programs completely independent of the processor it will be running on. It is the compiler software that understands the code and translates it to the processor-specific assembly code. They allow programmers to focus on the logic of the program rather than the hardware details. However, understanding low-level languages like assembly and machine language can provide valuable insights into how computers work at a fundamental level.

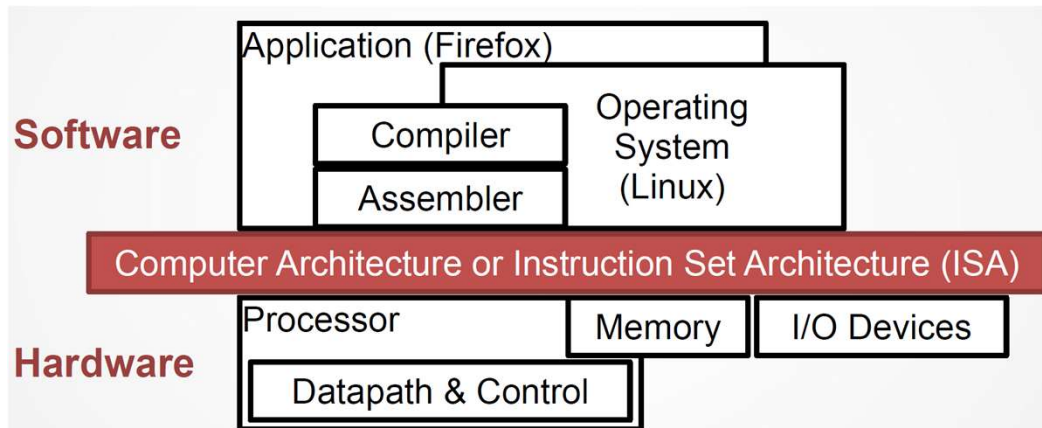
The animation shows the three different levels of a program starting from a short program written in high-level C code. The C code was then compiled which translated it into the corresponding assembly language specific to the

processor. Finally, the equivalent binary code for the program was also shown. This is the machine code equivalent of the instructions shown earlier.



The hardware/software interface

- We're currently at the transition point now! 😊



We're currently at the transition point now. We just made hardware and software connect.

Computer architecture serves as a crucial bridge between hardware and software design. The processor, which is the brain of the computer, operates by following a defined sequence: it fetches an instruction, retrieves the necessary operands, performs the specified operation, and writes back the result. This process is implemented through intricate networks of logic gates and flip-flops, transforming electrical signals into meaningful computations.

On the software side, programs are typically written in high-level languages like Python or C, which are more user-friendly for human programmers. These high-level instructions are then compiled into assembly language, which the processor can understand and execute directly. This compilation process translates human-readable code into machine-level instructions, allowing seamless interaction between software and hardware.

In essence, computer architecture harmonizes the design of hardware and software, enabling the efficient execution of complex programs by the processor.

This slide presents a comprehensive overview of the hardware-software interface in a computer system.

At the bottom, we have the fundamental building blocks of a computer: the processor, which is created using logic gates, flip-flops, and memory. I/O interfaces are also present here. These components form the physical layer of a computer, also known as the hardware.

In the middle, we have the computer architecture or instruction set architecture (ISA). This is the interface between the hardware and the low-level software. It defines the supported data types, the registers, and the hardware support for managing input/output mechanisms. We're currently at this stage now, where we are bridging the gap between a hardware implementation (i.e. the processor hardware) and the software implementation (i.e. the programs that we are running on the processor).

Above ISA, we have the system software which includes the assembler (translating assembly programs into machine code) and compilers (translating high-level language code into assembly language). We can then write more complex programs leading to the operating system, which manages hardware resources and provides services for the execution of several other software applications. And finally, we have applications, which are high-level software written in high-level languages. These are the programs that perform specific tasks based on user requirements.